

Your AI woke up in Vegas with no memory of last night. Build AI that doesn't forget with Cognee's self-hosted, hybrid graph-vector or memory layer. \$10,000...

PASTED

I am participating in a hackathon ->
<https://www.wemakedevs.org/hackathons/cognee>

Deliberated how to scaffold hackathon project support

Nice — that's a fun one, and the "AI woke up in Vegas with no memory" framing is a good hook to build around. Since Cognee is required tech, the strongest submissions usually lean hard into actually using the memory lifecycle (`remember()`, `recall()`, `improve()/memify`, `forget()`) rather than just calling `remember()` once and treating it like a vector DB.

What would be most useful for you right now?

Q: Where are you in the process?

A: checkout about it and get info of all

Weighed repository research against providing comprehensive overview

Here's the full picture, pulled together from the hackathon page and current info on Cognee itself.

The Hackathon

Cognee Hackathon by WeMakeDevs · **Jun 29 – Jul 5, 2026** (so you're mid-hackathon right now) · **\$10,000 in prizes** + job interview opportunities at Cognee.

- **Theme:** completely open — build anything (agent, app, game, tool, automation) as long as it uses Cognee for memory. The listed "tracks" on the overview page are just inspiration, not hard categories.

 Incognito chat

assigned → wait for assignment → submit PR. Max 5 PRs/person, low effort or AI spam PRs get rejected and repeat offenders banned.

- **IP:** belongs to your team.
- **Disclosure:** using AI assistants (Claude, Copilot, etc.) is allowed but must be declared in your submission.

What Cognee actually is

Cognee is an open-source AI memory platform for agents, giving AI agents persistent long-term memory across sessions with a self-hosted knowledge graph engine. The newest version simplified the stack a lot — the entire memory layer can now run on a single Postgres instance instead of separately-managed graph DB, vector DB, Redis, and relational DB, with retrieval moving between similarity search and graph structure without crossing service boundaries. You can still swap in dedicated backends like Neo4j or Neptune for graphs, and Qdrant, ChromaDB, Weaviate, or Milvus for vectors if you want. [GitHub + 2](#)

Core lifecycle (this is what the hackathon rules want you leaning on):

- `remember()` — ingest text/files/URLs into the knowledge graph (older API used `add()` + `cognify()`)
- `recall()` — query memory (older API: `search()`)
- `improve()` / `memify()` — post-ingestion enrichment, pruning stale nodes, adapting from feedback
- `forget()` — surgically delete/prune data

Minimal example pattern:

```
python

import cognee, asyncio

async def main():
    await cognee.add("Cognee turns documents into AI memory.")
    await cognee.cognify()
    await cognee.memify()
    results = await cognee.search("What does Cognee do?")

asyncio.run(main())
```

calls. An LLM_API_KEY is mandatory unless you point at a local Ollama or HuggingFace model, and there's an MCP server available so Claude Code, Cursor, etc. can read/write Cognee memory, with per-client auto-scoped datasets by default (toggleable). GitHub + 2

There's also a **hosted Cloud** option — free Developer plan (\$35 value) via code COGNEE-35 at platform.cognee.ai — useful if you don't want to self-manage infra during the hackathon.

Resources to actually use

What	Link
Discord (help/teammates)	discord.com/invite/m63hxKsp4p
Self-Improvement Quickstart	docs.cognee.ai/guides/self-improvement-quickstart
Full docs	docs.cognee.ai
Main repo	github.com/topoteretes/cognee
Karpathy Wiki example (real Cognee project)	github.com/topoteretes/karpathy-wiki
Company Brain hackathon guide	cognee-hackathons repo
Claude Code integration	cognee-integrations repo

Want me to help brainstorm a concrete project idea (something that showcases the full lifecycle, not just remember + recall), or dig into the Open Source PR track to find a good first issue?

so we are going for the macbook track

Clarified MacBook track distinctions and prize criteria

That distinction matters a lot for how you architect this:

What "MacBook track" implies for your build:

- You self-host Cognee (not the hosted Cognee Cloud platform) — meaning you configure your own LLM provider, vector store, and graph store locally rather than relying on `platform.cognee.ai`.
- Judges will specifically weight "Best Use of Cognee" (how deeply you lean on `remember/recall/improve/forget`) and **technical excellence**, since "open source" here really means "you ran the OSS engine yourself and it shows."

The 6 judging criteria, since they apply regardless of track:

1. Potential Impact
2. Creativity & Innovation
3. Technical Excellence
4. Best Use of Cognee (depth of lifecycle API usage)
5. User Experience
6. Presentation Quality (demo, README, submission clarity)

A few things I'd nail down before you start coding:

get all context about it

Fetches: The Hangover Part AI: Where's My Context? | WeMakeDevs

Here's the full picture on the **MacBook track** ("Best Use of Open Source"):

The two grand prize tracks

Track	Prize	Requirement
 Best Use of Open Source ← yours	Apple MacBook Neo (one per team member, or cash equivalent)	Best build on self-hosted, open-source Cognee

Every member of a winning team gets the full prize — no splitting. Max team size 4.

Side tracks (stackable, not exclusive with the MacBook track)

- **Best Blogs** → Keychron mechanical keyboard (\$120). Write about your build/journey/how Cognition gives AI memory.
- **Social Buzz** → top 10 posts tagging @wemakedevs + @cognition_ get exclusive swag.
- **Open Source PR Track** → \$100/PR, top 20, separate from your main build entirely (contribute to github.com/topotere/cognition/issues).

What "self-hosted" means practically

Since you're going for open-source, **don't use Cognition Cloud** (platform.cognition.ai) — that free credit is for the iPhone track. You'll configure your own LLM provider, vector store, and graph store via docs.cognition.ai/setup-configuration/overview.

Core API you'll be judged on using deeply

```
python

import cognition

await cognition.remember("Doug is the groom. The wedding is Sunday.")
await cognition.remember(file="vegas_receipts.pdf")

answer = await cognition.recall("Where is Doug?")

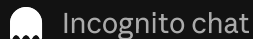
await cognition.improve() # a.k.a. memify

await cognition.forget(dataset="last_nights_mistakes")
```

(Note: docs elsewhere show the lower-level

`add()` / `cognify()` / `search()` / `memify()` calls —

`remember` / `recall` / `improve` / `forget` look like a friendlier wrapper introduced for this hackathon's branding. Worth checking docs.cognition.ai for which is current in your installed version.)



Idea categories (inspiration, not required)

Personal memory agents, research/knowledge copilots, never-forget workflows, self-improving agents, support/customer memory, learning tools — or something wilder.

Want to lock in a project idea now, or start with getting Cognee self-hosted locally (LLM provider, vector store, graph store choices)?

now i want deep research for this

so explore everything in deep because->

I want a idea for project/product that i will choose and make for this hackathon and want to win the hackathon, for this analyze also this->

1. Projects of previous winners of similar hackathons and whats special in their project and what's pattern
2. What currently cognee needs, what they want or purpose
3. while suggesting ideas make sure the ideas also matches with the hackathon track
4. And Find the idea which will get fundings in future but immediate target is to win hacakthon

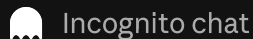
Charted comprehensive research strategy for hackathon project selection

Cognee hackathon idea research



Research complete • 342 sources • 7m 51s

Your Cognee "Hangover Part AI" Hackathon strategy report is ready. It covers Cognee's current positioning (post-1.0 release, \$7.5M seed, competitive landscape vs. Mem0/Zep/Letta), the reference projects Cognee promotes (Karpathy Wiki, Company Brain), and cross-cutting patterns that separate winning hackathon submissions from the rest. It includes a ranked shortlist of seven project ideas scored on Open Source track fit, Cognee lifecycle depth, differentiation, feasibility, and startup potential — with the Support Memory Copilot as the primary recommendation, plus a day-by-day build plan, alternative picks based on team strengths, and caveats on unverified data points.



so validate those ideas you said and this two also
and properly rank all

The Core Problem in Open Source Projects

Open source maintainers face a worse version of the internal-team problem, because:

Contributors come and go constantly — someone fixes a bug,

Show more

Reconstructed validation framework, acknowledged missing context, prioritized immediate rese...

Good additions. Before I merge everything into one ranked list, I need to flag something honest: I don't have the full text of the original 7 ideas retained verbatim in this window — only that "Support Memory Copilot" was the top pick. Rather than risk misdescribing your own report back to you, let me validate **CodeGuard** and **BrainOS** properly right now with fresh research, and you can paste the other 6 idea names/blurbs from the report (or tell me to regenerate them) so the final ranking is accurate rather than guessed.

Let me dig into competitive landscape and feasibility for these two first.

Weighed hackathon ideas against market saturation and judge expectations

Here's what the research shows — and it changes the picture a bit.

Validation: CodeGuard

The core concept is not new — but your specific angle is genuinely sharp.

"AI that gives a codebase institutional memory" is already a funded, competitive SaaS category: **Unblocked** (connects code to GitHub/Slack/Jira history, \$29/user/month, explicitly answers "why was this built this way"), **Swimm**, **Greptile**, **Sourcegraph Cody**, **Driver.ai**, and **Augment Code** are all live products doing versions of this. This isn't a gap in the market — it's a crowded one with real venture money behind it.

That said, none of these target **volunteer-run open source maintainers specifically**

separate from the actual Open Source PR bounty track (contributing real PRs) — don't let judges conflate the two.

Validation: BrainOS

This one I'd caution you against more strongly. The "company memory / AI coworker" category is not just crowded — it's dominated by very well-capitalized incumbents: **Glean** just raised \$150M at a **\$7.2B valuation** with \$300M ARR and 850+ employees, alongside Guru, Notion AI, Coveo, Moveworks, and Capacity all competing in the same "capture Slack/docs/tickets into queryable answers" space. A solo/small team cannot out-execute that in a week on capability or polish.

More importantly for *this specific hackathon*: Cognition's own reference materials already showcase almost exactly this pattern — their "Company Brain" hackathon guide and their own published customer-support use case ("Agent tracks past interactions, failed actions, resolved cases... retrieves similar resolved cases") are close cousins of BrainOS. If judges have already seen several "company brain" submissions in past Cognition events, this idea reads as safe-but-generic rather than differentiated. The fact that it's "easiest to build in 7 days" is often a tell that it's also the most obvious — and pattern research on hackathon winners tends to favor narrow-and-sharp over broad-and-familiar.

Ranked comparison (known ideas)

Idea	OSS track fit	Cognition lifecycle depth	Differentiation	1-week feasibility	Funding potenti
CodeGuard	Excellent — literally OSS maintainer tooling	High — natural remember/recall/improve/forget narrative	Medium — crowded category, but the self-referential Cognition-repo demo is a fresh spin <i>for this room</i>	Good — scoped data source (one repo), clear demo	Real cat (mainta tooling Tidelif, Sentry-adjacen compet with fui incumb
Support Memory Copilot	Good	High — directly mirrors Cognition's own published use case	Medium-low — closely follows Cognition's own	Good — narrow, well-	Crowde real (Interac

<i>pick — see caveat below)</i>			"independent insight"	scope	Sierra)
BrainOS	Good on paper	Medium — breadth over depth risks shallow demos of each lifecycle stage	Low — overlaps Cognee's own past "Company Brain" reference project, dominated by \$7B+ incumbents	Highest (as you noted) — but "easiest" often means "most generic"	Huge T but near odds of differen traction against Glean/C at this s

My ranking: CodeGuard > Support Memory Copilot > BrainOS.

CodeGuard wins on the sharpest, most memorable demo for *this specific room of judges* and the deepest natural use of the memory lifecycle. Support Memory Copilot is the safest bet for hitting "Best Use of Cognee" since it mirrors their own materials almost exactly — good if you want lower risk, weaker if you want to stand out. BrainOS I'd actually deprioritize despite it looking easiest — it's the idea most likely to blend into a pile of similar submissions.

One gap: I don't have the other 6 ideas from the original report retained in this session — I only carried forward "Support Memory Copilot" as the headline pick. Could you paste the other 6 blurbs from that report? Once I have them I'll fold them into this same scoring table for a complete, single ranked list rather than a partial one.

this was the report now validate and rank all properly

Winning the Cognee "Hangover Part AI" Hackathon — Open Source Track Strategy

TL;DR

- ****Build a self-improving, self-hosted "vertical brain" agent that visibly exercises Cognee's full memory lifecycle (remember → recall → improve/memify → forget) with a live knowledge graph visualization**

Incognito chat

AI agents built on agent-memory infrastructure).

- **Winners in this space are narrow, polished, and "feel like a product, not a hackathon project"** — they solve one real pain for one clearly-named user, show depth (multi-hop graph reasoning, feedback loops, tests), and nail the demo/storytelling. Broad "RAG chatbot" demos lose.
- **To win the Open Source track specifically:** self-host everything (local LLM via Ollama, LanceDB/pgvector, Kuzu/Neo4j), lean hard into the parts that differentiate Cognee from Mem0/Zep (graph + ontology + **memify** self-improvement + **forget**/GDPR), and make the memory graph the visible star of the demo.

Key Findings

What Cognee is and what it wants right now (July 2026)

- Cognee (Berlin-based, Topoteretes UG; founders Vasilije Markovic (CEO) and Boris Arzentar (CTO)) shipped **cognee 1.0** on June 26, 2026, restructuring its API around four verbs: **remember**, **recall**, **improve**, **forget** (legacy **add** / **cognify** / **memify** / **search** remain as lower-level building blocks). The hackathon's "Best Use of Cognee" criterion rewards depth across exactly these APIs.
- Cognee raised a **\$7.5M seed** announced February 19, 2026, led by **Pebblebed** (Pam Vagata, ex-OpenAI founding team; Keith Adams, ex-FAIR), with **42CAP**, Vermilion Cliffs Ventures, and angels from Google DeepMind, n8n, and Snowplow. It is **seed-stage** and in **growth mode**. Traction figures should be read carefully because Cognee's numbers jumped between funding announcements: the Feb 2026 seed coverage cited "more than 70 companies running cognee in live environments" and "crossed 12,000 GitHub stars with 80+ contributors," while the June 26, 2026 cognee 1.0 announcement claims **~6 million memories/month** created across 100+ companies and the site header shows ~17,500 stars. Named customers include Bayer (scientific research), University of Wyoming (policy evidence graph), Dilbloom, dlthub, and Knowunity. Its ECL pipeline ingests data from **38+ sources/formats**.
- Cognee's positioning vs. competitors is the key to "delighting" the team. **Mem0** (vector-first, personalization; **\$24M** across a Kindred-led seed + Basis Set-led Series A, Oct 28, 2025; **41,000** GitHub stars and 14M Python downloads — the largest community) and **Zep/Graphiti** (temporal knowledge graph) are the main rivals; **Letta** (formerly MemGPT) raised a **\$10M seed** led by Felicis at a reported **\$70M post-money** (Sept 2024; ~23K+ stars by June 2026). Cognee's differentiators are: **hybrid graph+vector+relational in one**

local deployment.**

- What Cognee promotes as flagship use cases (from cognee.ai, GitHub README, and blog): **Company Brain** (unify scattered company data), **customer-support memory** (org/agent/user memory tiers, cited answers), **SQL/expert-knowledge copilots**, **persistent codebase memory for coding agents** (Claude Code plugin, [codify](#) AST parsing), **research/knowledge wikis** (the Karpathy Wiki), and **workflow memory**. Vasilije's stated research frontiers: **adaptive retrieval via trace optimization**, temporal/"predictive memory" (work with Berkeley/Brown), and enabling **multi-agent shared state** ("the next hard problem... getting many agents to work together across tools, teams, and systems that were never designed to share state").

Reference projects Cognee points hackers to

- **Andrej Karpathy Wiki** ([andrej-karpathy.com](#); [topoteretes/karpathy-wiki](#)) — a markdown-first, Cognee-backed self-growing wiki built by CEO Vasilije Markovic. Every answer cites its source and is filed back as a new page ("asking a question is the same as editing the wiki"); strict citation linting ("claims have to point at a real source file or the linter complains"). It exercises recall, search, ingest, improve, and lint. This is the clearest "reference of taste" for what Cognee finds impressive: **compounding memory + strict provenance/citations + a template others can reuse**.

- **Company Brain** ([topoteretes/cognee-hackathons](#), [cognee-companybrain-hackathon-2026-06-16](#)) — an official "hackathon guide and reference for building a company-wide memory with Cognee," unifying data from various company sources (finance, support, product history) into one graph, then reconstructing timelines, retrieving similar resolved cases, and updating memory after execution. (Exact connector list/code structure could not be verified via automated fetch — open the repo directly to confirm specifics.)

Cross-cutting patterns that separate winners from average submissions

Drawing on the hackathon's own rubric (Impact; Creativity & Innovation; Technical Excellence; **Best Use of Cognee**; UX; Presentation) and evidence from comparable agent-tooling hackathons (GitLab AI Hackathon 2026, Dust/Qonto, Microsoft AI Agents, Agno Global Agent Hackathon):

1. **"Feels like a product, not a hackathon project."** In the GitLab AI Hackathon 2026, the standout project **LORE** (a "Living

was the recap's headline project.) Polish, tests, and a real README win.

2. **Narrow + specific beats broad + generic.** Winners pick one named user and one painful workflow (B2B sales intent, supply-chain risk, code security review) rather than "an assistant that remembers everything."

3. **Show the mechanism the sponsor cares about.** GitLab winners leaned into knowledge graphs because that's what GitLab is building; for Cognition, that means visibly exercising the **graph + memify self-improvement + forget** — not just vector recall (which reads as "just RAG").

4. **Demo storytelling and provenance.** Winning agent demos show a before/after, cite sources, and prove state persists across a fresh session. The Karpathy Wiki's "every claim cites a real source or the linter complains" is the model.

5. **Solve a real pain for a real audience** — judges consistently reward projects that map to a budget owner (support teams, GRC/compliance, sales ops, dev teams).

2026 funding backdrop (why some ideas have "legs")

- **Vertical AI agents are the dominant investable category**: per New Market Pitch's 2026 agentic-AI funding analysis, **"Vertical AI Agents remain the main investable surface... They account for 48.3% of 2026 year-to-date deals and 54.6% of capital,"** because investors can underwrite "a specific buyer, budget, and outcome." Bessemer's **"Building Vertical AI"** playbook argues: **"Vertical AI isn't competing for IT budgets; it's competing for labor budgets... Vertical AI taps directly into the labor line of a P&L. That's why we believe Vertical AI represents a fundamentally larger opportunity than vertical SaaS ever did."** Median agentic round in 2026 YTD is **~\$19M** (average ~\$36M — skewed by mega-rounds).

- **Agent memory infrastructure is a funded but structurally thinner category**: Mem0 (\$24M), Letta (\$10M seed), Zep, Supermemory, Memories.ai, and Cognition itself have all raised. But New Market Pitch flags that standalone **Agent Memory Systems** have the weakest capital-to-deal ratio (~0.30x) — "memory is necessary but not yet a large independent profit pool." As Mem0 CEO Taranjeet Singh puts it, **"Every agentic application needs memory, just as every application needs a database."**

- **Implication**: the ideas most likely to attract post-hackathon interest (VC or a Cognition job offer) are **vertical agents with proprietary compounding data + a memory moat** (attach memory to a



Each idea is scored on fit to the Open Source track (self-hostable), "Best Use of Cognee" (lifecycle depth), differentiation, week-1 feasibility, and hackathon-to-startup potential.

#1 — Support Memory Copilot ("Never Asks Twice")

****What:**** A self-hosted customer-support agent with three isolated memory tiers — ****organization**** (product docs, policies, resolved-case playbooks), ****agent**** (learned resolution strategies), and ****user**** (per-customer ticket/interaction history). On each ticket it recalls similar resolved cases, answers with citations, and after resolution runs `improve()/memify` so it never repeats a failed fix.

****Lifecycle depth:**** remember/add (docs + tickets), cognify (case/entity graph), recall/search (GRAPH_COMPLETION cited answers), memify/improve (feedback reweighting), forget (GDPR delete a customer). Uses per-session/per-user isolation — a real Cognee feature (`get_sessionized_cognee_tools`).

****Why it wins:**** It IS Cognee's #1 promoted use case (their own org/agent/user support-memory blog post + README scenario), exercises every API, and demos beautifully (a customer returns; agent recalls full history and cites the prior fix).

****Startup legs:**** Highest. Vertical support-agent + memory moat is squarely in the top-funded 2026 category and hits the "labor-budget" thesis; strongest signal to Cognee (mirrors their own GTM).

****Feasibility:**** High — Cognee ships most of the primitives; a small team can polish UX in a week.

#2 — Living Team Brain / Knowledge-Continuity Engine

****What:**** A self-hosted "company brain" that ingests Slack, Google Drive/Notion, meeting transcripts, and git history into one graph, then answers "what's the status of Acme?" or "why did we choose Postgres?" with cited, timeline-aware answers. Directly modeled on Cognee's Company Brain reference and GitLab's headline LORE project.

****Lifecycle depth:**** Heavy remember/cognify across many connectors; recall with temporal reasoning (`temporal_cognify`); memify to prune stale facts and reweight; forget for offboarding.

****Why it wins:**** Maps to the Company Brain reference; strong Impact narrative (knowledge walks out the door when people leave — LORE's exact winning pitch).

****Startup legs:**** Strong — competes with Glean but self-hosted/graph-native; enterprise budget owner.

****Feasibility:**** Medium — connector breadth is the risk; scope to 2-3

 Incognito chat

error knowledge base"), using Cognition's `codify` AST parsing and the Claude Code plugin. The agent recalls prior fixes, performs dead-code/dependency analysis via the call graph, and improves from each session.

****Lifecycle depth:**** `codify/cognify` (AST → graph), recall (multi-hop over call graph), `memify` (learn from resolved bugs), `forget` (per-service dataset namespacing).

****Why it wins:**** Deeply technical (Technical Excellence) and directly advances a Cognition roadmap item (extending `codegraph` beyond Python to JS/TS, Java, Go, Rust) → strongest ****hireability**** signal.

****Startup legs:**** Medium-high — dev-tools + memory; also the best "Cognition wants to hire you" play, given interview slots are a headline prize.

****Feasibility:**** Medium — leverage the existing Claude Code plugin; scope language support tightly.

#4 — Right-to-be-Forgotten / Provable Memory Erasure (Compliance)

****What:**** A GDPR/DPDP "Article 17" erasure layer that proves an agent forgot a user across graph + vector + logs, issuing a signed, offline-verifiable deletion receipt. (A community project along these exact lines already exists under the Cognition GitHub topic.)

****Lifecycle depth:**** Showcases the under-demoed `forget()` / `prune` — a genuine differentiator vs. vector-only tools — plus `remember/cognify` to build the graph being erased.

****Why it wins:**** Novel/creative; enterprise-compliance impact; almost nobody else will demo `forget()` deeply.

****Startup legs:**** Medium — compliance is a real budget; narrower TAM but defensible.

****Feasibility:**** High — narrow, well-defined scope.

#5 — Personal Research Wiki Engine (Karpthy Wiki, generalized)

****What:**** A self-hosted, self-growing knowledge wiki template: ingest papers/docs/URLs, every answer cites sources and is filed back as a page, strict citation linting. Point it at any domain.

****Lifecycle depth:**** `remember/recall/improve` with a visible compounding loop; strong provenance.

****Why it wins:**** Matches Cognition's own reference project of taste; great UX/presentation; "compounding memory" is exactly the narrative Cognition sells.

****Startup legs:**** Lower (closer to a tool/template than a company) but an excellent portfolio/interview piece.

best-action from outcomes.

****Lifecycle depth:**** Full lifecycle; temporal reasoning for deal-state changes.

****Why it wins:**** Clear budget owner, strong Impact; Cognee explicitly lists "a sales team's deal memory" as a target use case.

****Startup legs:**** Strong — vertical sales AI is heavily funded.

****Feasibility:**** Medium — use synthetic CRM data for the demo.

#7 — Multi-Agent Shared Memory Orchestrator

****What:**** Several specialized agents (researcher, writer, reviewer) reading/writing one shared Cognee graph, demonstrating cross-agent knowledge sharing and no-repeat learning — directly targeting Vasilije's stated "next hard problem: many agents sharing state."

****Lifecycle depth:**** Shared remember/recall + memify across agents; the most forward-looking use.

****Why it wins:**** Highest Creativity/vision; aligns with Cognee's stated future direction.

****Startup legs:**** Medium — infra-flavored, ambitious.

****Feasibility:**** Lower — hardest to make robust in a week; real risk of an unpolished demo.

Recommendations

****Primary pick: build #1 (Support Memory Copilot).**** It is the best-balanced bet across all six judging criteria and the strongest hackathon-to-startup pipeline. Concretely:

- **Days 1-2:**** Self-host the stack (Ollama local LLM, LanceDB or pgvector, Kuzu/Neo4j). Get `remember → recall` working on synthetic support tickets + product docs. Stand up per-user/per-org isolation.
- **Days 3-4:**** Add the `memify/improve` feedback loop (thumbs-up/down on resolutions reweights the graph) and `forget()` (GDPR-delete a customer live on stage). Build a ****graph visualization**** panel — make the memory the visible star.
- **Days 5-6:**** Polish UX, write a real README with an architecture diagram, and record a tight demo showing (a) a customer returning and the agent recalling full history, (b) the agent citing sources, (c) it learning from a correction, (d) a fresh session proving persistence. Add a handful of tests (the LORE "43 tests" signal that judges read as product maturity).
- **Day 7:**** Submit + write the required blog post framing the pain ("support teams don't have a support problem — they have a context problem," Cognee's own line).

demoed `forget()` API.

- If the team is **design/UX-heavy and small (1-2 people)** → **#5 (Wiki Engine)** is the highest-polish-per-effort and lowest technical risk.

- **Avoid** generic "chatbot that remembers" builds and anything that only uses vector recall — that reads as "just RAG" and will score low on "Best Use of Cognee."

Cross-cutting winning tactics (apply to any pick):

- Self-host end-to-end and say so loudly (own LLM/vector/graph) — this is the literal Open Source track requirement.

- Exercise **all four verbs** and show `memify` and `forget` explicitly (most teams will skip these two — differentiate there).

- Make the **knowledge graph visible** and every answer **cited** (Cognee's aesthetic).

- Frame Impact around a **named budget owner**; keep scope narrow and the demo polished.

Caveats

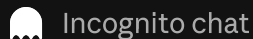
- **No public winners list exists for Cognee's own past AI-Memory hackathons** (Berlin June 2026; SF with Pebblebed). The "reference projects of taste" (Karpathy Wiki, Company Brain) are Cognee-authored templates, not necessarily prior winning submissions. Named Cognee-using hackathon projects (Tendrill, Uh Oh!, Fortify DD) come from a *separate* third-party event (Bright Data's "Web Data UNLOCKED"), so treat them as ecosystem signal, not direct precedent.

- The exact connector list and code structure inside the **Company Brain** reference repo could not be fully verified (GitHub blocked automated fetch); confirm by opening the repo directly before relying on specifics.

- This is a **new hackathon** (first WeMakeDevs × Cognee event, June 29–July 5, 2026), so winner patterns are inferred from analogous agent-tooling hackathons and the published rubric, not from prior Cognee-track results.

- Cognee's traction figures are inconsistent between its Feb 2026 seed announcement (12,000 stars / 70+ companies) and its June 2026 cognee 1.0 post (17,500 stars / 100+ companies / 6M memories per month); verify against the live repo/site before quoting.

- Funding-trend figures (median round sizes, category shares, market projections) come from VC/industry analyses and include forward-looking projections; treat market-size numbers as estimates, not certainties. GitHub star counts and funding details for competitors were cross-checked but shift frequently.



Consolidating nine ideas and reconciling duplicate concepts

Good — now I have the full field. Let me validate the factual anchors and check competitive crowding on the ideas I haven't fact-checked yet (2, 4, 6, 7), since the picture changes meaningfully once you look at what's already out there.

Synthesized nine hackathon ideas, ranked Support Memory Copilot first, CodeGuard second

Now I have real validation data on all the gaps. Here's the complete, fact-checked ranking of all 9 ideas.

What changed after validation

- **Cognee's funding is accurate as reported:** \$7.5M seed, led by Pebblebed (Pam Vagata, ex-OpenAI; Keith Adams, ex-FAIR), Feb 19, 2026 — confirmed across Cognee's own blog, Pebblebed's portfolio page, and PitchBook (total raised ~\$9M across two rounds).
- **CodeGuard's category is crowded** (Unblocked, Swimm, Greptile, Sourcegraph, Augment Code, Driver.ai all do "why was this built this way" for codebases) — but none target volunteer OSS maintainers specifically, and the self-referential demo angle is a real differentiator *for this room*.
- **BrainOS/Living Team Brain is the single most duplicated idea in the whole set** — it's not just competing with Glean (\$7.2B valuation, \$300M ARR) and Guru externally, it's also functionally identical to Cognee's own published "Company Brain" hackathon reference project. Any team building this is very likely rebuilding what Cognee already showed hackathon participants how to build.
- **Right-to-be-Forgotten is more differentiated than the original report gave it credit for.** Fresh research confirms this is a genuinely hot, unsettled 2026 problem — EU AI Act high-risk obligations phase in August 2026, CCPA's ADMT rules phase in January 2027, and law firms/compliance blogs are actively writing about how agent memory breaks standard GDPR Article 17 deletion. Competition exists (one academic framework, one early-stage startup called Chanl) but it's nowhere near as saturated as the "company brain" or "codebase Q&A" categories.
- **Multi-agent shared memory is a genuine open research problem** — 2026 papers confirm consensus/write-conflict handling in shared agent memory is

Rank	Idea	Cognee lifecycle depth	Differentiation	1-week feasibility	Funding/
1	CodeGuard	High — natural remember/recall/improve/forget arc	Medium-high — crowded category, but the <i>judge-personalized</i> demo (point it at Cognee's own repo, live) is unmatched for this specific room	High — scoped to one public repo	Medium - real market but funded by strongest incumbent; an interview signal
2	Right-to-be-Forgotten	High — the only idea that puts <code>forget()</code> center stage, which almost no one else will demo	High — validated as the least crowded space in the whole set, with real regulatory urgency	High — narrow, well-defined scope	Medium - real compliance budget, timely
3	Support Memory Copilot	High — mirrors Cognee's own published use case almost exactly	Medium-low — safe, but reads as "the docs example, rebuilt"	High	Medium-low — crowded but real (Intercom, Decagon, Sierra)
4	Self-Improving Coding-Agent Memory	High — technically the deepest, extends Cognee's actual roadmap (Claude Code plugin, codify)	Medium — dev-tools space has entrants, but this is literally advancing	Medium	Medium-low — best path "get hired by Cognee" for

 Incognito chat

	Shared Memory Orchestrator	Cognee CEO's stated research frontier	genuinely unsolved problem, few working demos exist	risk of a live demo breaking (write-conflict handling is an open research problem)	impresses vision, high variance
6	Personal Research Wiki Engine	Good but narrower (light on forget)	Medium-low — directly clones Cognee's own Karpathy Wiki reference artifact	High	Low — close to a tool/temp than a company
7	Sales/Deal Memory Agent	Good	Low — GTM AI is brutally crowded (Gong, Clari, Salesforce Agentforce, Attention) and least tied to anything Cognee specifically promotes	Medium	Nominally strong category, unrealistic differentials odds
8=	Living Team Brain	Medium — breadth over depth risk	Lowest — collides with Glean (\$7.2B), Guru, Notion AI, and Cognee's own Company Brain reference project	High (deceptively — "easiest" here usually means "most generic")	Large TAI near-zero realistic differentials

above

above

Bottom line

If I had to pick one: CodeGuard. It's the only idea where the demo itself is built for the specific people judging it — showing Cognee's own repo remembering its own maintainers' decisions is the kind of moment that's hard to forget in a room full of submissions. It also naturally threads the hackathon's dual structure (main track + the Open Source PR bounty track) without you having to explain why it fits.

Close second: Right-to-be-Forgotten, if your team would rather own an underused, timely, and much less crowded technical angle than compete on demo charisma.

Actively deprioritize Living Team Brain / BrainOS — it's the one idea in this set most likely to collide with what other teams submit, since it's the closest match to Cognee's own most-publicized reference project.

Want me to sketch the actual day-by-day build plan and architecture for CodeGuard (or Right-to-be-Forgotten) the way the original report did for Support Memory Copilot?

now git codegaurd what things we need? which are free which are paid?

Architected CodeGuard's cost breakdown: free stack versus paid alternatives

Here's the full shopping list for building CodeGuard, split by what actually costs money and what doesn't. Good news: you can build the entire thing for **\$0** if you make a couple of smart choices.

What you need, piece by piece

open source
(Apache-2.0)

Data source	Pull issues/PRs/comments from Cognee's repo	GitHub REST/GraphQL API — free with a Personal Access Token, 5,000 requests/hour authenticated. Plenty for ingesting a few hundred issues/PRs in a week	—
Vector store	Semantic search over memories	LanceDB (embedded, no server, Cognee's default) or pgvector on a local Postgres	Managed pgvector/Pinecone if you want hosted — not needed here
Graph store	The knowledge graph itself	Kuzu (embedded, zero-setup, Cognee supports it directly) or NetworkX for something even simpler	Neo4j AuraDB free tier exists (200K nodes/400K relationships) but adds setup overhead you don't need. Neo4j Community Edition self-hosted is also free if you want the nicer graph visualization tooling
LLM (the one real cost driver)	Entity/relationship extraction during cognify , plus answering questions during recall	Groq (free tier, very fast, hosts Llama/Qwen models) or Google Gemini (Flash models have a genuine free tier via AI Studio)	OpenAI (gpt-4o-mini) or Anthropic Claude — cheap per-token but not free. cognify is LLM-call-heavy (one extraction call + several embedding calls per chunk), so a full ingest of Cognee's issue/PR history

			extraction quality
Embeddings	Turn text into vectors	Local via <code>sentence-transformers</code> (HuggingFace, free) or Ollama's <code>nomadic-embed-text</code>	OpenAI <code>text-embedding-3-small</code> — fractions of a cent per 1K tokens, cheap but not free
Frontend/demo UI	What judges actually see	Streamlit (fastest to build in Python) or plain HTML+FastAPI	—
Hosting for a live link	Optional, for a shareable demo URL	Render free tier — 750 hrs/month, free static hosting, sleeps after 15 min idle (fine for a hackathon demo). Or skip hosting entirely and demo from your laptop with <code>ngrok</code> (free tunnel)	Render/Railway paid tiers only matter if you need it running 24/7 past the hackathon
Version control/submission	Required for judging	GitHub repo — free	—

Important nuance on "self-hosted"

The Open Source track requires you **don't route through Cognition Cloud** — it doesn't require your LLM to be self-hosted too. "Own LLM provider" just means bring your own API key (Groq, Gemini, OpenAI, Anthropic, or local Ollama all count) rather than using `platform.cognition.ai`'s managed backend. So you don't need a GPU or local model to qualify — a free-tier Groq or Gemini key is completely legitimate for this track.

My recommended \$0 stack for CodeGuard specifically

 Incognito chat

- + Groq free-tier API (LLM extraction + recall)
- + GitHub PAT (data ingestion)
- + Streamlit (frontend)
- + run locally, ngrok for live demo during judging

This costs nothing, sets up in under an hour, and cleanly satisfies "self-hosted, own LLM provider" since nothing routes through Cognition Cloud. If partway through the week you find Groq's free tier too slow or rate-limited for the volume of GitHub issues/PRs you're ingesting, that's the one place worth spending a few dollars on OpenAI or Anthropic — everything else on this list has no reason to ever cost money for a hackathon-scale build.

we are gonna build in TS mostly
where will host that bot hosted?

Weighed hosting options for dual-component TypeScript architecture

Good news — Cognition just shipped a **first-class official TypeScript SDK** (`@cognition/cognition-ts`) as part of the 1.0 release, so you can build the whole thing in Node/TS without touching Python. That changes the architecture slightly, and it affects hosting.

The real architecture (two pieces, not one)

The TS SDK is a client — it talks to Cognition's actual memory engine, which runs as a service via Cognition's official Docker image (`cognition/cognition` on Docker Hub, exposing the REST API on port 8000). So you have:

1. **The Cognition engine** — runs as a container, holds the vector store + graph store + does the `remember/recall/improve/forget` work
2. **Your bot** — a Node/TypeScript app (e.g. built with **Probot** or plain **Octokit**, both TS-native) that listens for GitHub webhooks (new PR, new issue comment) and calls the Cognition engine over HTTP for answers

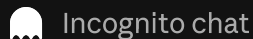
Both need to be reachable, and — important detail — since GitHub webhooks fire in real time whenever something happens on the repo, if you want CodeGuard to

Where	Good for	Free?
Render	Best all-in-one pick — runs your Node bot as a free Web Service (750 hrs/month) <i>and</i> can deploy the <code>cognee/cognee</code> Docker image as a second service, both from one dashboard	Yes, free tier covers both. Sleeps after 15 min idle — fine, GitHub webhooks wake it on the next event, just adds a few seconds of cold-start lag
Railway	Cleanest developer experience, one-click Docker Compose deploy for both services together	No real free tier anymore in 2026 — \$5/month Hobby plan minimum. Worth it if your team doesn't mind the small spend for reliability
Fly.io	Good if you want your bot always-on with no cold starts, supports persistent volumes for your graph/vector data	Has a usage-based free allowance, but check current limits before committing — it's shifted around over the past year
Local + tunnel (ngrok/Cloudflare Tunnel)	Zero cost, works fine if you're doing an <i>interactive</i> demo where you drive the queries live in front of judges	Free, but the public URL isn't stable across restarts, which is annoying for a GitHub App webhook that needs to sit registered for days — not great if you want it silently listening all week

My recommendation

Go with **Render**: deploy the `cognee/cognee` Docker image as one Web Service, and your Probot/Octokit TypeScript bot as a second Web Service that talks to it over Render's internal networking. Both fit in the free tier for a week-long hackathon, it's genuinely \$0, and you get a stable public URL the whole time — which matters if part of your demo is "look, it already caught something on its own this week" rather than only answering questions live.

One thing worth deciding now: do you want CodeGuard to be a **live, always-**



we have the azure subscription so how the things work?
like for web we can use vecel from users can deploy bot for their use
they can upload docs related to it and all etc

Architected multi-tenant Azure stack integrating Cognition, PostgreSQL, and Vercel

This actually simplifies things nicely — Cognition has **first-class native Azure OpenAI support** (it's a documented provider, not a workaround), and Azure's Postgres offering can run the *entire* memory layer in Cognition 1.0's newer, simplified architecture. Your Azure subscription is a genuinely good fit here, not just "also possible."

The clean split: Vercel for the front door, Azure for the memory engine

Vercel — your multi-tenant web app (Next.js, TS-native, generous free tier). This is where users sign in, connect their GitHub repo, and upload extra docs (coding standards, decision logs, whatever). Vercel is exactly right for this piece — don't overthink it.

Azure — everything that actually does the remembering:

Piece	Azure service	Why it fits
Cognition engine	Azure Container Apps	Runs the official <code>cognition/cognition</code> Docker image directly. Consumption plan scales to zero when idle — cheap for a hackathon-scale workload
Vector + graph + metadata store	Azure Database for PostgreSQL - Flexible Server (with <code>pgvector</code> extension enabled)	Cognition 1.0 can run its <i>entire</i> memory layer — vector, graph, and relational — on a single Postgres instance now, instead of needing separate graph/vector servers. Azure's managed Postgres does this in one resource
LLM + embeddings	Azure OpenAI Service	Cognition has dedicated config for this out of the box — <code>LLM_MODEL="azure/gpt-5-mini"</code> , <code>LLM_ENDPOINT</code> , <code>LLM_API_KEY</code> , <code>LLM_API_VERSION</code> . Same pattern for embeddings (<code>azure/text-embedding-3-large</code>). No

listener

execution grant, TypeScript-native

About the multi-tenant "users upload their own docs" feature

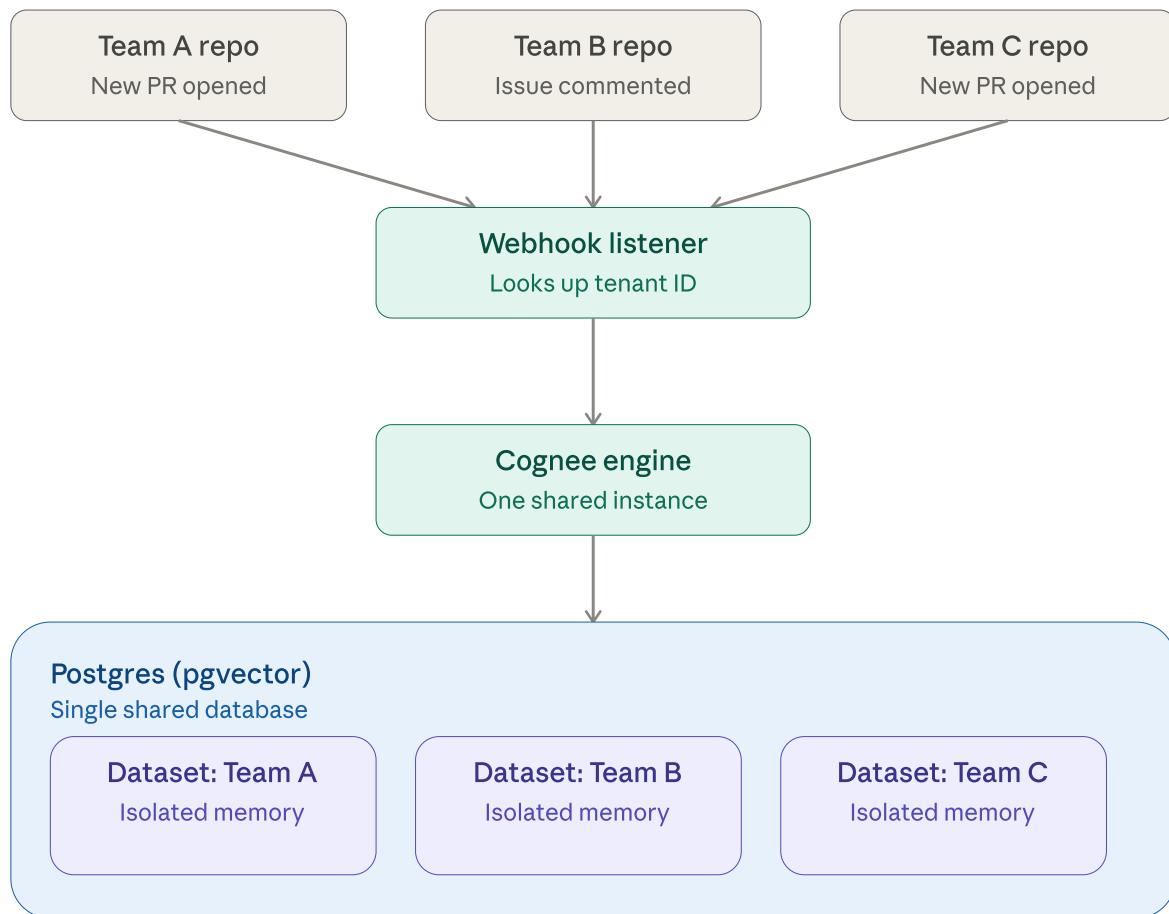
This is very achievable, because Cognee actually ships **built-in multi-tenancy** — dataset-level read/write/delete/share permissions per user, confirmed working across Postgres, Neo4j, Kuzu, and LanceDB. In practice that means each user's connected repo + uploaded docs live in their own isolated dataset inside the *same* Postgres instance — you don't need to spin up separate databases per customer, Cognee's session/dataset scoping handles the isolation for you. That's the hard part of multi-tenant SaaS mostly solved for free.

Free vs. paid on Azure specifically

- **Azure Container Apps** — has a real monthly free grant (roughly 180K vCPU-seconds / 360K GiB-seconds), enough to run a light always-on service for a hackathon week
- **Azure Database for PostgreSQL Flexible Server** — has a free-tier-eligible burstable instance under most student/hackathon credit programs
- **Azure Functions** — generous free execution grant, essentially free at hackathon scale
- **Azure OpenAI** — this is the one piece that's genuinely pay-as-you-go with no standing free tier, but it's exactly what your Azure credits are for — this is the intended use case for subscription credits, so lean on it here rather than hunting for a separate free LLM provider

One honest flag: this has grown from "point CodeGuard at Cognee's repo for a demo" into "a real multi-tenant SaaS with auth, GitHub OAuth, and a doc-upload UI." That's a stronger pitch for the "fundability" angle we discussed, but it's meaningfully more surface area to build in a week. Want me to help scope a cut-down MVP version of this (what's truly needed for judging day vs. what can be a "coming soon" slide), or are you set on building the full multi-tenant version?

but we are hosting 1 bot how does it will work for different users



The trick is that "one bot" only means one running server process — it doesn't mean one shared memory. Everything that differs between users lives in the database, not in the server:

- Every GitHub webhook carries an `installation.id` and `repository.full_name` in its payload. That's how the listener knows *whose* event just arrived.
- Your own small mapping table (installation ID → dataset name) tells the listener which Cognee dataset belongs to that tenant.
- The listener calls Cognee scoped to that one dataset — `recall(query, dataset="team_a")` — never a global query across everyone's data. Cognee's dataset permissions genuinely isolate at the graph and vector level, not just a soft filter, so Team B's bot can never surface Team A's memory even though they're hitting the same server and the same Postgres instance.
- The reply gets posted back using that installation's own GitHub token, so it only ever touches the repo it came from.

Onboarding a new user, concretely: they install your GitHub App on their repo (standard GitHub OAuth flow) → that fires an `installation.created` webhook you catch once → you generate their dataset ID and store the mapping → if they upload extra docs on your Vercel dashboard, those get `remember()`'d straight into that same new dataset → from then on every event from their repo routes there automatically.

why use azure for everything forcebally, use wherever actually needed, and we are not getting free alternate, like hosting engine, maybe bot and all

Weighing LLM costs against free hosting alternatives strategically

Fair pushback — I was defaulting to Azure because you mentioned having the subscription, not because it's actually the best fit for most of these pieces. Let me redo this properly: use Azure only where it genuinely helps, and real free options everywhere else.

Evaluated cost-effective platform alternatives and consolidated stack strategically

Fair call — I was reaching for Azure by default just because you mentioned having it, not because it was the best fit. Here's the honest breakdown, piece by piece.

What actually needs to be paid for vs. what doesn't

Piece	Genuinely free option	Does Azure add anything here?
Frontend	Vercel free tier	No — same thing, no reason to use Azure
Bot/webhook listener	Vercel serverless functions — same platform as your frontend, one less thing to manage	No — Azure Functions does the same job with zero advantage
Cognee engine (the Docker container)	Google Cloud Run — genuine Always Free tier: 2 million requests/month, 360,000 GiB-seconds, 180,000 vCPU-seconds,	No — Azure Container Apps has a comparable grant, but Cloud Run's free tier is more generous and there's no reason to prefer one over the other if both

 Incognito chat

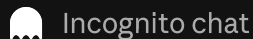
Postgres + pgvector store	Neon free tier — 100 compute-hours/month, scales to zero after 5 min idle and wakes in ~1 second (good fit for a hackathon week with bursty usage), and it's the database that actually powers Vercel Postgres, so it plugs in natively	No — Azure's Postgres Flexible Server free tier exists, but Neon is simpler to wire up and you're already on Vercel
LLM calls (cognify + recall)	Groq or Gemini free tier — genuinely \$0	Yes, this is the one place your Azure subscription actually earns its keep. Not because Azure OpenAI is cheaper than anyone else — it isn't, it's the same pay-per-token model as OpenAI direct — but because if you burn through Groq/Gemini's free-tier rate limits mid-week, you already have real credits sitting there to fall back on

The actual recommendation

Vercel (frontend + webhook listener) + **Google Cloud Run** (Cognee engine) + **Neon** (Postgres/pgvector) + **Groq or Gemini free tier** (LLM). That's \$0, spread across two platforms instead of four, and nothing sits idle waiting for you to configure IAM roles or resource groups.

Keep Azure OpenAI in your back pocket, not in the critical path — only reach for it if the free-tier LLM providers start rate-limiting you during a heavy ingestion run. That's a real, legitimate use of the subscription. Using Azure for the database or the container hosting just because the credits exist would be solving a problem you don't have.

does cognee allows for the google cloud what there track is asking the open source track of the macbook prize what we can do?



Good — this checks out cleanly, and it's worth being precise about because it matters for judging.

The distinction the hackathon is actually drawing

It's not "cloud vs. no cloud." It's **which software does the memory work:**

- **Cognee OSS** — the actual open-source engine (`pip install cognee` or the `cognee/cognee` Docker image). You configure it, you choose the LLM/vector/graph backends, you control it. This is "self-hosted" regardless of what hardware it physically runs on.
- **Cogwit** — Cognee's own managed cloud product. *They* run the Kuzu/LanceDB/Postgres clusters and orchestration for you, and you just call their API with a key — no infrastructure decisions on your end.

The Open Source track's requirement is: use the first one, not the second. That's it.

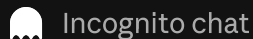
What that means for your stack

Deploying the open-source `cognee/cognee` Docker image on Google Cloud Run is **fully self-hosted and 100% compliant with the Open Source track**. Google Cloud is just the landlord providing compute — the actual memory engine running inside that container is still the open-source software you configured yourself. Same logic applies to Neon (self-managed Postgres+pgvector, not Cognee's managed store), and to Groq/Gemini/Azure OpenAI (your own LLM provider, not Cognee's bundled one).

What would actually disqualify you from this track is if your app called `cognee.serve(url="https://your-instance.cognee.ai", api_key="ck_...")` — that's routing through Cogwit, Cognee's managed service, which is the *other* track's territory (the iPhone one).

So the stack we landed on — Vercel + Cloud Run + Neon + Groq/Gemini, with Azure OpenAI as backup — is exactly right for the Open Source track. Nothing about using major cloud providers for hosting works against you here; the judges care that *you* own the memory layer's configuration, not that you're running it off a physical box under your desk.

If you want to make this bulletproof for the judges, it's worth a single line in your README: "Self-hosted Cognee OSS, own LLM/vector/graph config — not using



Postgres + pgvector store Neon free tier — 100 compute-hours/month, scales to zero after 5 min idle and wakes in ~1 second (good fit for a hackathon week with bursty usage), and it's the database that actually powers Vercel Postgres, so it plugs in natively No — Azure's Postgres Flexible Server free tier exists, but Neon is simpler to wire up and you're already on Vercel

we are using best ai database also free?

Reconciled Neon's free tier advantages against specialized vector database alternatives

Good instinct to double-check this one — and there's actually a technical reason it matters more than "which vector DB is fastest," which I should have flagged the first time.

The real question isn't "best vector database" — it's "what actually persists on Cloud Run"

Cognee's *default* stores — LanceDB (vector) and Kuzu (graph) — are file-based. They write to local disk. That's perfect for a laptop or a single always-on VM. But Cloud Run containers are stateless and ephemeral: they scale to zero, spin up fresh, and don't guarantee the local filesystem survives between requests or cold starts. If we deployed Cognee with its defaults on Cloud Run, memory could silently vanish between one user's webhook event and the next — a bad bug to discover during judging.

So we don't actually have a choice between "fast dedicated vector DB" vs. "good enough Postgres" — we need *some* external, persistent database no matter what, because the deployment target requires it. The real question is which one.

Why Neon/pgvector is still the right pick, not just the easy one

Cognee 1.0's newer architecture lets the *entire* memory layer — vector, graph, and relational metadata — live in one Postgres instance instead of three separate systems. That matters a lot for us specifically:

- **Multi-tenancy is only confirmed for four backends: pgvector, Neo4j, Kuzu, and LanceDB.** Qdrant — despite being a genuinely excellent, purpose-built vector database — isn't on that list. Since CodeGuard's whole architecture depends on per-tenant dataset isolation, picking a vector DB Cognee doesn't

 Incognito chat

to track, one connection string — real time savings in a one-week build.

- **pgvector's HNSW indexing is genuinely competitive** at hackathon scale (thousands to low millions of vectors — nowhere near where dedicated vector DBs start to matter).

So to directly answer it: yes — for *this* project, Neon + pgvector isn't a compromise, it's the correct choice, not just the free one. A dedicated "AI-native" vector database (Qdrant, Zilliz, Weaviate) would solve a scaling problem we don't have while creating two we do: an extra service to host, and no confirmed multi-tenant isolation in Cognee.

One thing worth confirming while you're setting this up: double check in Cognee's config docs that `graph_database_provider` can actually be set to `pgvector` / Postgres in the current release for the *graph* layer too (not just vector) — the "single Postgres instance" claim was from Cognee's own announcement, but I'd verify the exact env var before you're mid-build, since if graph-on-Postgres isn't fully wired up yet, you'd fall back to Kuzu for the graph specifically — which would then need its own persistent volume mount on Cloud Run rather than local ephemeral disk. Want me to dig into that config detail so you're not discovering it the hard way on day 2?

okay then make proper plan now with what project is what it gonna have what things will be using all

Synthesized comprehensive hackathon plan with scoped features and timeline

Let's lock this in as a real plan you can execute against, accounting for where you actually are in the clock — today's July 1, hackathon ends July 5, so you've got about 4.5 days of build time left.

What CodeGuard is

An institutional-memory bot for open-source repos. It remembers every past maintainer decision, PR rejection, and "why is this built this way" reasoning — then automatically catches new contributors re-proposing something already tried and rejected, and answers onboarding questions instantly instead of a maintainer re-explaining it for the 50th time. Flagship demo: point it at Cognee's own GitHub repo

 Incognito chat**P0 — the core demo, must be solid**

1. Ingest Cognee's GitHub repo (issues, PRs, comments, maintainer decisions) via `remember()`
2. `recall()` — answer a real contributor question with a cited past decision
3. Live catch — new PR/issue opens on the watched repo → bot checks it against memory → comments if it matches something already rejected
4. `improve()` / memify — reweights so frequently-cited decisions surface first, stale ones decay
5. `forget()` — demoed live, wiping a dataset's derived memory. Almost nobody else will show this API, so it's cheap differentiation
6. One rehearsed live moment: open a PR that resembles a documented past rejection, watch CodeGuard catch and comment on it in real time

P1 — only if P0 is solid with days to spare

7. Multi-tenant onboarding — any user installs the GitHub App on their own repo, gets an auto-isolated dataset
8. Doc upload via a dashboard (style guides, ADRs) into that dataset
9. A dashboard to browse ingested memory / graph stats

Be honest with yourselves about this split as the week goes on. A rock-solid single-repo demo beats a wobbly multi-tenant one in judging, every time.

Final tech stack

Piece	Choice	Notes
Language	TypeScript, <code>@cognee/cognee-ts</code>	Official SDK, no Python needed in your app code
Cognee engine	<code>cognee/cognee</code> Docker image on Google Cloud Run	Always Free tier, scales to zero
Store	Neon Postgres + pgvector	Single instance for vector + relational; confirm graph-on-Postgres support on day 1 — fallback is Kuzu if it's not fully wired up in the current release
LLM + embeddings	Groq/Gemini free tier primary	Use your Azure credits only if free tier

App installs for P1

Frontend/dashboard	Vercel (Next.js)	Only strictly needed for P1 — P0 can demo via PR comments + terminal output
Data source	GitHub REST/GraphQL API, personal access token (P0) or App installation token (P1)	5,000 req/hr free, plenty for a repo-scale ingest
Multi-tenancy	Cognee's native dataset-level permissions	P1 only

Day-by-day (today = Jul 1)

Day	Focus
Jul 1 (today, rest of day)	Environment setup: deploy <code>cognee/cognee</code> on Cloud Run, wire up Neon, confirm one <code>remember()</code> → <code>recall()</code> round trip works end to end with your chosen LLM key. Pull a sample slice of Cognee's issues/PRs via the GitHub API into a local file — just prove the data access works, don't ingest yet
Jul 2	Build the real ingestion pipeline — walk Cognee's issues/PRs/comments, <code>remember()</code> them into a dataset. Run it for real on a meaningful slice (a few hundred items — enough for a rich graph, not so much you blow through free LLM rate limits). Validate <code>recall()</code> gives good cited answers on a handful of test questions
Jul 3	Build the live-catch flow: webhook listener → <code>recall()</code> → decide → comment via GitHub API. Test end to end on a scratch repo before trusting it on the real one. Wire <code>improve()</code> to run post-ingestion
Jul 4	Polish, add the <code>forget()</code> demo moment, write the README (state "self-hosted, not Cogwit" clearly) and the required blog post. If there's real time left, start P1. Record a backup demo video in case live network issues hit on judging day
Jul 5 (submission day)	Final test pass, rehearse the demo script out loud at least twice, submit with buffer before the deadline

cited answer

3. The centerpiece: trigger a pre-staged PR that resembles a documented past rejection → CodeGuard's comment appears live, citing the exact prior decision
4. Quick `forget()` moment — "and we can prune this on request, which almost nobody demos"
5. Close on the differentiator: full memory lifecycle, self-hosted, reading its own judges' repository right now

One open item to close out today, not later: confirm whether Cognee's current release actually supports `graph_database_provider=postgres`, or whether you need Kuzu for the graph piece specifically. That single config decision affects your Cloud Run setup, so it's worth nailing down before you build anything on top of it. Want me to dig into that now?

create proper markdown file with plan, info, how things should be built and all

Synthesized hackathon project details into comprehensive markdown documentation

This has everything we've worked through — the track requirements, feature scope split into P0/P1, the full tech stack with the free-vs-paid reasoning, the day-by-day schedule, demo script, and the open risk on graph-store support that's worth resolving today before you build further on top of it. Good luck with the build.



Codeguard build plan
Document · MD

Download

